A Student Information System that refuses to lie

Every claim in this talk is a measurement, or it is not in the talk.

Sujith · UCS3513 System Design Laboratory
github.com/sujith0613/sis-system-design

10 minutes · live demo

THE POINT

Everyone demos the happy path.

Ours demos the failure path.

Redis. MongoDB. Rate limits. Every project in this room has that diagram.

This one can prove it — so it can also show you what happens when it breaks.

THE STARTING POSITION

Three claims, and how each one dies

Each of these is true of a prototype that never has to fail in public.

“Rate limited”

Two replicas.
Two counters.
Double the limit.

“Sharded”

One process.
One dictionary.
hash(key) % 3.

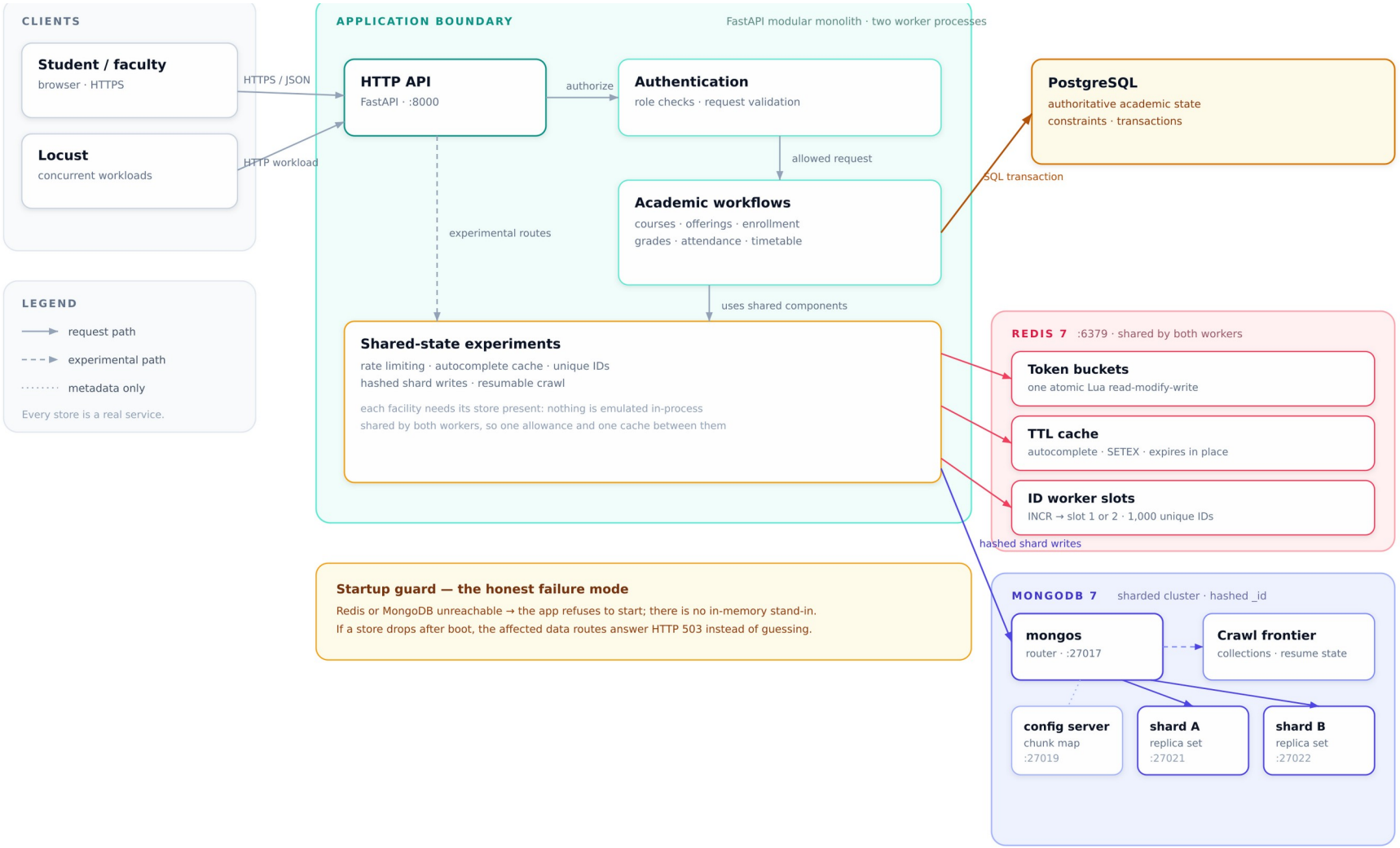
“Cached”

Cache and truth share
a lifetime, so
neither is real.

We deleted all three. No flag, no fallback: if a store is down, the system refuses to answer.

ARCHITECTURE

Three real stores, one startup guard



Six containers on one host: PostgreSQL, Redis, config server, two shards, mongos. Nothing is emulated in-process.

THE UNIT'S CHECKLIST

Eight components, each with a job you can observe

COMPONENT	WHAT IT DOES HERE	WHERE THAT STATE ACTUALLY LIVES
Cache	autocomplete results with server-side expiry	Redis · TTL 20 s, allkeys-lru
Unique IDs	time-sortable 64-bit identifiers	Redis worker slots
Rate limiter	one quota shared by every process	Redis · atomic Lua
URL shortener	create and redirect (307)	PostgreSQL
Web crawler	robots-aware frontier, resumable	MongoDB collections
Autocomplete	prefix search over the catalogue	Redis in front of PostgreSQL
Consistent hashing	100 virtual nodes, routing shown	application ring
Sharding	hashed shard key, real chunk moves	MongoDB · 2 shards + mongos

Each one answers on a live endpoint under /system-design/ — not on a slide.

CLAIM 1

It refuses to run on a lie

Take Redis away and the honest answer is 503 with the store named.

A per-process fallback would have answered 200 — and silently doubled every client's quota. Availability loses to honesty here, on purpose.

Recovery is the store coming back — not a restart.

AT BOOT

```
$ uvicorn app.main:app
RuntimeError: Required state stores
  are not available:
  - Redis unreachable at :6399
ERROR: Application startup failed.
```

AT RUNTIME, WITH REDIS KILLED

```
$ podman stop sis-redis   $ curl /courses
[503]  {"status": "degraded",
        "store": "redis",
        "detail": "…cannot decide…"}
```

CLAIM 2

Our hash ring disagrees with the database — in public

THE ROUTING DISAGREEMENT, AS SERVED

```
$ curl /system-design/route/DEMO-KEY-1
{ "ring_prediction": "sis-shard-b",
  "actual_owner":    "sis-shard-a",
  "agrees": false }
```

THE MOVEMENT IS REAL

```
$ curl -X POST …/move-chunk
{ "from_shard": "sis-shard-a",
  "to_shard":   "sis-shard-b" }
```

Two answers, one endpoint.

The left number is the consistent-hashing ring from the lecture. The right is what mongos actually does with a hashed shard key. They disagree, and the API says so with a boolean.

The ring routes. moveChunk moves. One of those is a drawing; the other changed 129 documents between two replica sets on this machine.

CLAIM 3

The invariants hold when things go wrong

5 / 20

seats under concurrency

One 5-seat offering, 20 simultaneous enrolments: 5 x 201, 15 x 409, enrolled_count = 5.

3 → 0

one bucket, two workers

Worker A spends all three tokens; worker B is refused.

1,000

IDs from two slots

Two processes never mint the same number: 1,000 generated, 0 collisions.

0

pages refetched

A second crawl took 5 URLs off the persisted frontier and fetched none twice.

The shared thing lives in the store, not in the process.

Every number here came out of the running stack, and the suite asserts the counts, not the vibe.

What this is not

- One host. No failover, no backup, no replication test.
- Redis is volatile on purpose: a restart resets the buckets.
- Crawler is polite: 3 pages in 7.1 s at 0.5 s delay.
- Load numbers predate the migration, so they are not quoted.
- One cache flag was deleted, never defaulted to off.
- The suite wipes the dev database — a rough edge, not a secret.

A demo you cannot re-run is a story.

`./run.sh`
checks the machine, starts six containers, seeds the data, polls /health

`./stop.sh`
stops it, then proves every port is closed

github.com/sujith0613/sis-system-design